

PCI - passthrough with KVM/libvirt qemu/ virtmanager

A PCI passthrough tutorial using libvirt and KVM on GNU/Linux

Introduction

This tutorial covers some of the nuances involved in setting up PCI passthrough with libvirt and KVM / QEMU. The documentation contained herein is primarily sourced from personal experience and research into the topic.

Since I don't have any compatible AMD®-based hardware for testing, I can only provide in-depth support for Intel®-based builds. However, the steps outlined in this publication should very accurately reflect the workflow required for AMD®-based systems.

So if you're having difficulties passing through some pcie devices to a kvm virtual machine, this guide will help you

Preamble

The goal of this tutorial is to set up a KVM hypervisor environment capable of PCI passthrough. This tutorial assumes that the would-be hypervisor's operating system and libvirt/KVM have already been installed.

Also of great importance to this tutorial is to provide the reader (you) with ample information to understand the changes that are made and *why* they need to be made.

Several quirks will present themselves during configuration and each one must be overcome in order to complete all of our stated goals:

Hardware Compatibility Criteria

Below are some things to keep in mind while planning your build. At a minimum, your hardware must meet all criteria with bold text. Other criteria are just recommendations to make the configuration process easier.

- If you're using an AMD®-based system, your motherboard and CPU **must** support [AMD® Virtualization Technology \(AMD-V™\)](#) and [AMD® I/O Virtualization Technology \(IOMMU\)](#) (also known as "AMD-Vi").
- If you're using an Intel®-based system, your motherboard and CPU **must** support [Intel® Virtualization Technology \(VT-x\)](#) and [Intel® Virtualization Technology for Directed I/O \(VT-d\)](#).

Preparing the Hypervisor

Step #1

The first step to facilitating PCI-e passthrough of any kind is enabling virtualization acceleration and IOMMU in your motherboard's BIOS settings. Consult your motherboard manual to find options relating to AMD-V™, AMD-Vi, VT-d, VT-x or anything else with virtualization-related keywords and enable those options.

Next, we need to enable support for IOMMU in the Linux kernel. Assuming that you use GRUB as your boot loader, we need to append one of the following strings to the `GRUB_CMDLINE_LINUX_DEFAULT` variable in `/etc/default/grub`, making sure that there is a space to separate it from what's already there:

- If using an AMD®-based system, append `amd_iommu=on` to the kernel argument list. If
- using an Intel®-based system, append `intel_iommu=on` to the kernel argument list.

Once you've modified the variable, it should look similar to this:

```
GRUB_CMDLINE_LINUX_DEFAULT="intel_iommu=on"
```

Note: If your device(s) don't support interrupt remapping, you might be able to quell any issues with PCI-e passthrough by adding `vfio_iommu_type1.allow_unsafe_interrupts=1` to the argument list as well. There are some security concerns that you should research first before enabling unsafe interrupts. ^[Red Hat]

Finally, run the following command to update your boot loader configuration:

```
$ update-grub
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-x.xx.x-xx-generic
Found initrd image: /boot/initrd.img-x.xx.x-xx-generic
done
```

Once your boot loader configuration is updated, reboot your system. To verify that your operating system is configured correctly for PCI-e passthrough, run the following commands and make sure that your output is similar:

```
$ apt-get install -y cpu-checker
```

```
$ kvm-ok
```

```
INFO: /dev/kvm exists
```

```
KVM acceleration can be used
```

```
$ dmesg | egrep 'DMAR|IOMMU'
```

```
[ 0.000000] ACPI: DMAR 0x00000000BD6483E0 0000B8 (v01 INTEL BDW 00000001 INTL 000
[ 0.000000] DMAR: IOMMU enabled
[ 0.000000] DMAR: Host address width 39
```

```
[ 0.000000] DMAR: DRHD base: 0x000000fed90000 flags: 0x0
[ 0.000000] DMAR: dmar0: reg_base_addr fed90000 ver 1:0 cap c0000020660462 ecap f0101a
[ 0.000000] DMAR: DRHD base: 0x000000fed91000 flags: 0x1
[ 0.000000] DMAR: dmar1: reg_base_addr fed91000 ver 1:0 cap d2008c20660462 ecap f010da
[ 0.000000] DMAR: RMRR base: 0x000000bdea5000 end: 0x000000bdeb3fff
[ 0.000000] DMAR: RMRR base: 0x000000bf000000 end: 0x000000cf1ffffff
[ 0.000000] DMAR-IR: IOAPIC id 8 under DRHD base 0xfed91000 IOMMU 1
[ 0.000000] DMAR-IR: HPET id 0 under DRHD base 0xfed91000
[ 0.000000] DMAR-IR: x2apic is disabled because BIOS sets x2apic opt out bit.
[ 0.000000] DMAR-IR: Use 'intremap=no_x2apic_optout' to override the BIOS setting.
[ 0.000000] DMAR-IR: Enabled IRQ remapping in xapic mode
[ 0.700747] DMAR: No ATSR found
[ 0.700776] DMAR: dmar0: Using Queued invalidation
[ 0.700782] DMAR: dmar1: Using Queued invalidation
[ 0.700828] DMAR: Setting RMRR:
[ 0.700883] DMAR: Setting identity map for device 0000:00:02.0 [0xbf000000 - 0xcf1ffffff]
[ 0.702040] DMAR: Setting identity map for device 0000:00:14.0 [0xbdea5000 - 0xbdeb3fff]
[ 0.702083] DMAR: Setting identity map for device 0000:00:1a.0 [0xbdea5000 - 0xbdeb3fff]
[ 0.702128] DMAR: Setting identity map for device 0000:00:1d.0 [0xbdea5000 - 0xbdeb3fff]
[ 0.702141] DMAR: Prepare 0-16MiB unity mapping for LPC
[ 0.702174] DMAR: Setting identity map for device 0000:00:1f.0 [0x0 - 0xffffffff]
[ 0.702290] DMAR: Intel(R) Virtualization Technology for Directed I/O
[ 1.146736] [drm] DMAR active, disabling use of stolen memory
```

Note: you may get more or less lines here; just look for hints that IOMMU is actually enabled.

Step #2

Load the necessary modules for pcie passthrough in kvm/libvirt qemu/virtmanager at boot up

(or manually, but at boot up/automatically is preferred, obviously)

We can achieve this in a few ways...

Method 1: editing, or adding modules name to `/etc/modules-load.d/modules.conf`

Method 2: using systemd: this is called a systemd service file /or systemd unit file /or systemd service unit file

Method 1:

1. Add components necessary to load VFIO (Virtual Function I/O). VFIO is required to pass full devices through to a virtual machine, so that Ubuntu loads everything it needs. Edit and add the following modules names to **`/etc/modules-load.d/modules.conf`** file:

```
vfio
vfio_pci
vfio_iommu_type1
kvm
kvm_intel
pci_stub
```

Method 2: using systemd; create a systemd service file to load necessary modules at boot up

create a systemd service file and make it load the necessary modules at boot up:

```
sudo modprobe vfio
sudo modprobe vfio_pci
sudo modprobe vfio_iommu_type1
sudo modprobe kvm
sudo modprobe kvm_intel
sudo modprobe pci_stub
```

If everything is configured correctly, the virtual machine should now be able to boot and use the attached PCIe device without issue. All that remains now is the installation of the appropriate device drivers if necessary.

Glossary

passthrough devices—A set of devices which are the target of PCI-e passthrough and each neighboring device within their respective IOMMU groups. This set should exclude any “PCI bridge” devices which are part of the CPU.